

Modeling the Goodness of Fit for the Equations of Motion for a Spring Pendulum from Small Angle Approximations

Karina Winn
Alexander Blad
Kylee Thomas

Background

The spring pendulum is an interesting variation of a simple pendulum. In two dimensions, this system has two degrees of freedom. We will define these degrees of freedom as ϕ (the angle measured from vertical) and r (the length of the spring). Equations of motion for this system can be found by first writing equations for kinetic and potential energy, then using those to find the Lagrangian equations for the system.

$$\begin{aligned} -mgr\sin\phi &= mr^2\ddot{\phi} + 2mr\dot{r}\dot{\phi} \\ mr\dot{\phi}^2 + mg\cos\phi - kr &= m\ddot{r} \end{aligned}$$

These equations are coupled and therefore cannot be solved analytically. However, if r is replaced with $l + \epsilon$, where l is the equilibrium length of the spring and ϵ is the displacement of the spring, then small angle approximations can be used to uncouple the equations, which gives the following:

$$\begin{aligned} -mg(l + \epsilon)\sin\phi &= m(l + \epsilon)^2\ddot{\phi} + 2m(l + \epsilon)(\dot{l} + \dot{\epsilon})\dot{\phi} \\ m(l + \epsilon)\dot{\phi}^2 + mg\cos\phi - k(l + \epsilon) &= m(\ddot{l} + \ddot{\epsilon}) \end{aligned}$$

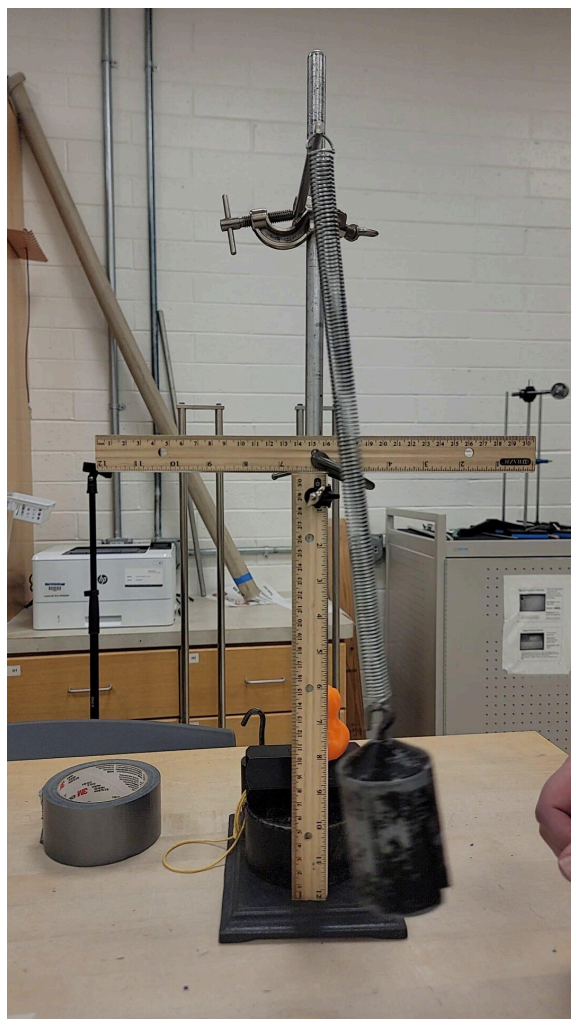
Solving both of these differential equations will give equations for ϕ and ϵ as functions of time.

$$\begin{aligned} \phi(t) &= A\cos(\sqrt{\frac{g}{l}}t - \delta) \\ \epsilon(t) &= B\cos(\sqrt{\frac{k}{m}}t) + C\sin(\sqrt{\frac{k}{m}}t) \end{aligned}$$

The purpose of this experiment is to experimentally verify these equations and see if they accurately describe the system.

Collecting Data

This is the apparatus that was designed to collect our data.



It consists of a spring and a mass which make the spring pendulum. Affixed to the pendulum are two rulers which set the scale for the reference frame of the position of the mass, which will be used by the video analysis software in LoggerPro to return an x and y position. The origin of the coordinate system is at the base of the vertical ruler. The x and y position data is used to determine the angle that the spring pendulum makes with the vertical axis (the ruler). Initially, the procedure was meant to measure the angle directly using a protractor, however after the first few attempts to collect data it became clear that this was not a viable method for data collection, thus the conversion from x and y position to angle.

The x and y position data was collected by taking a video at 15 frames per second. The number of frames was chosen so that the difference in position frame by frame would be significant enough to see a change in the position of the pendulum

without being so large that the data becomes unusable, or so small that the position in the pendulum seems unchanging. (For example, 1 second at 240 frames per second would give a lot of data points, but basically no useful information) Multiple runs of the pendulum were recorded and then analyzed using LoggerPro video analysis. The data given from the video analysis was then exported to a CSV and parsed through using Python (mainly the pandas library). One thing to note is that the initial conditions for each run may have been slightly different, which would greatly impact the motion of the system over time if the spring pendulum were a chaotic system. However, this approach assumes that the pendulum oscillates through only small angles, and therefore would not be chaotic in nature. The presumed error in this case would be the standard deviation of the position for each run at the measured time.

Error Analysis

The error analysis for this project was a bit tricky to figure out. There were many factors to consider in the approach taken for the experiment. One source of error within the experiment was from the video analysis software that was used to collect the data points. Based on specification from the developer website, the error from the software is less than one percent, so it was assumed that considering all other sources of error, this factor was negligible and can be assumed to effectively be zero. With that said, the rest of the error was estimated to be due to uncertainty in the measuring devices used (the rulers with 1mm increments), and the distortion from the wide angle lens of the camera used to record the motion of the spring pendulum. This error was propagated by estimating the uncertainty of the measurements taken, $\sigma_x = \pm 5$ mm and $\sigma_y = \pm 10$ mm. Because the x and y positions were converted into angles, the propagation of error for was found through the following.

$$\sigma_\phi = \sqrt{\sigma_x^2 * \frac{\partial \phi^2}{\partial x} + \sigma_y^2 * \frac{\partial \phi^2}{\partial y}}$$

For the stretch/compression of the spring, ϵ , a similar method was used to estimate and propagate the error. As mentioned earlier, if the behavior of the system turns out to be chaotic in nature, the model may not even be valid. However, moving

forward with the assumptions made this should sufficiently account for the error of the measurements of the system.

Fitting Methods

In order to test the goodness of fit for the data collected it was decided that a gradient search to optimize the parameters of the fit function would be used. The gradient search algorithm “searches” through hyperparameter space following the descending gradient of some fit function until it reaches a minimum. In this case, the optimization function is χ^2 due to the usefulness of χ^2 in determining the goodness of fit. By optimizing χ^2 it should ensure that the parameters found through the gradient search will be a good fit for the fit function. Initially the two fit functions were

$$\phi(t) = A \cos(\sqrt{\frac{g}{l}}t - \delta)$$

$$\epsilon(t) = B \cos(\sqrt{\frac{k}{m}}t) + C \sin(\sqrt{\frac{k}{m}}t)$$

but after an initial analysis of the data it became clear that these functions did not fit the data well, returning very large values for χ^2 as well as very low values for the correlation coefficient r . Based on that analysis, two new fit functions were determined.

$$\phi(t) = \left(A - B \cos(Ct) \right) \cos\left(\sqrt{\frac{g}{l}}t - \delta \right)$$

$$\epsilon(t) = \left(A - B \cos(Ct) \right) e^{-\beta t} \cos\left(\sqrt{\frac{k}{m}}t - \delta \right)$$

These two functions were determined to be the new fit functions based on the observed behavior of the collected data as seen on the following graphs.

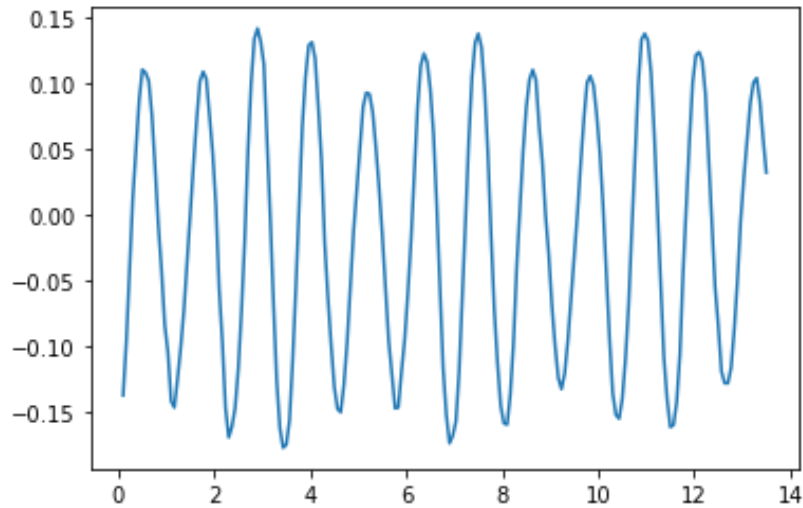


Fig 1) Plot of $\Phi(t)$

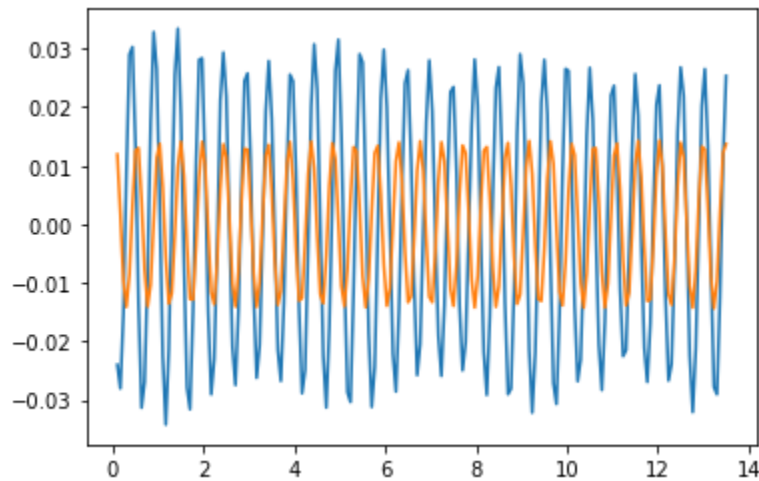


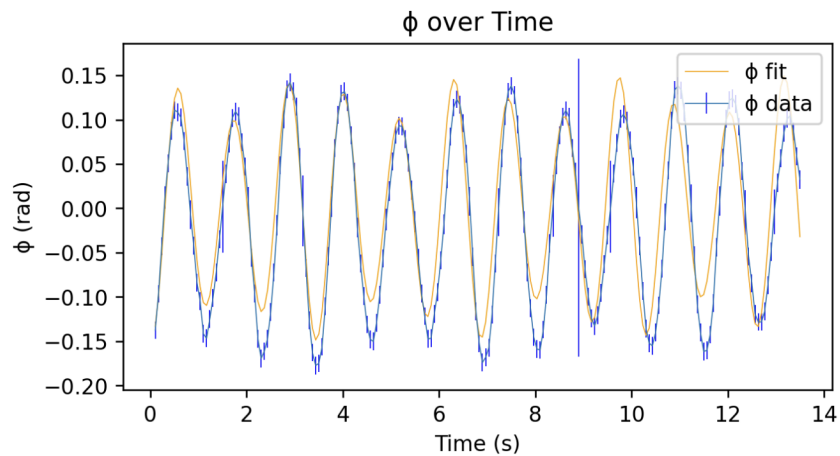
Fig 2) Plot of $\epsilon(t)$ with original fit function

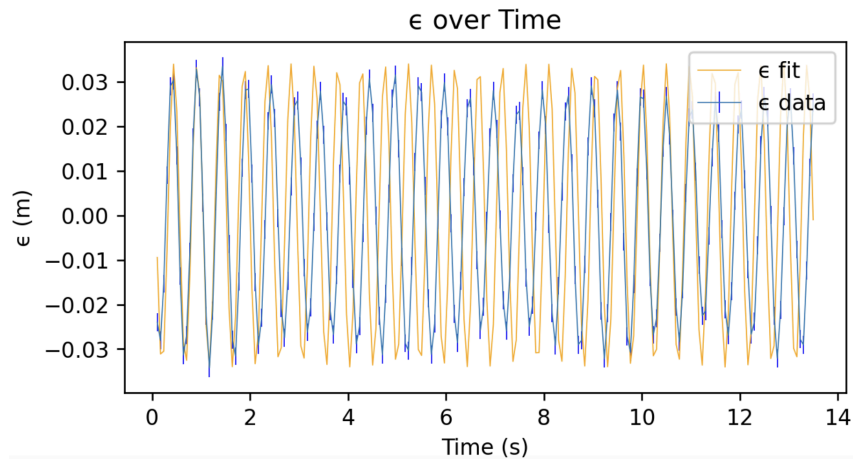
In fig 1, there appears to be some additional oscillatory motion that is not accounted for in the initial fit function for $\Phi(t)$. Something similar can be seen in fig 2. Fig 2 shows that there is some damping in the oscillation of $\epsilon(t)$ which caused the amplitude of the function to decrease over time (as show in the figure the amplitude in the original fit function, the orange colored line, is constant whereas the actual measured data is most certainly not). Further analysis was then done using these new fit functions and is discussed below. After running the gradient descent algorithm, the last three points of χ^2 calculated were used to fit a parabola in order to find a more accurate minimum and a more accurate value for each parameter.

It must be said that although the methods used are accepted and commonly used, one issue with the gradient descent method is its limitation to local minima. If the algorithm finds a local minima before it finds the global minimum of the hyperparameter space it will stop there, which leads to inaccurate values for the parameters being optimized. This was something that became quite apparent throughout the duration of this project, due to the complexity of the χ^2 hyperparameter space of the fit functions. We hypothesize that due to the oscillatory motion, as well as the potential chaotic nature of the system if the angles oscillated through are not sufficiently small, led the gradient descent algorithm to get stuck at multiple different local minima as it was optimizing each parameter. This oversight may have led to some of the issues faced while conducting this experiment.

Results

Running the gradient descent method gave the parameters of the fit functions for ϕ and ϵ described above. For the ϕ equation, the calculated parameters are as follows: $A = -0.125 \pm 1.586\text{e-}10$, $B = 0.260 \pm 6.345\text{e-}10$, $C = 1.900 \pm 9.518\text{e-}09$, $I = 0.325 \pm 6.345\text{e-}11$, and $\delta = 0.098 \pm 3.173\text{e-}10$. This fit has a χ^2 value of 13.825 and a correlation coefficient of 0.923. For the ϵ equation, the calculated parameters are: $A = 0.040 \pm 4.731\text{e-}11$, $B = 0.001 \pm 9.463\text{e-}12$, $C = 0.030 \pm 4.731\text{e-}11$, $\beta = 4.380\text{e-}5 \pm 2.366\text{e-}14$, and $\delta = -0.400 \pm 9.463\text{e-}10$. This fit has a χ^2 value of 51.386 and a correlation coefficient of 0.175. The previously measured I value was 0.261 ± 0.002 m, which is not within the error range of the I value from the ϕ fit equation. That combined with χ^2 values that are mediocre at best means that the functions do not fit the data.





There is however, another issue with these results. When the parabolic fitting was done with the last three parameter and χ^2 values from the gradient search, a couple of the parameters had a major problem. That is, the parabolas created from the B parameter in the ϕ equation and the A and δ parameters in the ϵ equation had a positive curvature. This means that there was no new minimum χ^2 , so it's hard to tell if the values for those parameters are valid.

Future Experiments

There are a few things that could be done to take the next step were this project to be continued. One very important thing would be figuring out why some of the parabolas there were created when optimizing the parameters had positive curvature, leading to inaccurate values which greatly impacted the fit of functions to the data that was collected. Improving the optimization and analysis process would be an excellent step in the right direction for further refining of this project.

Additionally, improvements to the data collection process would be required to move forward. Although the method used to collect data initially worked, there are tools available to make data collection not only more accurate, but less time consuming as well. One such method could be the use of a microcontroller attached to a gyroscope/accelerometer. This would allow for collection of positional data in 6 axes instead of just the 2 that were measured (not to mention that angular positional data

could be measured directly!). This would also allow for better estimation of the uncertainty and therefore more accurate propagation of error.

Improvements could also be made in the optimization of the parameters of the fit function for ϕ and ϵ . It is clear that as it is now, the gradient descent algorithm that was used to optimize the parameters of the fit functions works, but not very well. Whether this is due to the complexity of the hyperparameter space and its many local minima, or due to the implementation of the optimization algorithm there is much improvement to be made. Perhaps a better approach would be some hybridization of other optimization techniques, or perhaps a refined implementation of the gradient descent itself would be sufficient.

A final interesting avenue to explore is the nature of the system itself. Is the system truly harmonic, or is there chaos that is introduced when the angles oscillated through are not sufficiently small? Based on very limited observation it appears that there may be some chaotic motion for $\epsilon(t)$ but not for $\Phi(t)$. An interesting path to follow in the future.

Bibliography

Bevington, P., & Robinson, D. K. (2003). *Data Reduction and Error Analysis for the Physical Sciences*. McGraw-Hill Education.

Kósa, Á. (2011, March 7). *2D Spring Pendulum*. Wolfram Demonstrations Project.

Retrieved April 8, 2024, from

<https://demonstrations.wolfram.com/2DSpringPendulum/>

Morin, D. (n.d.). Chapter 6 - The Lagrangian Method. Retrieved March 27, 2024, from

<https://scholar.harvard.edu/files/david-morin/files/cmchap6.pdf>

Taylor, J. R. (2005). *Classical Mechanics*. University Science Books.

Code

```

from numpy import
    array, append, ones_like, sqrt, linspace, arange, cos, sin, dot, arctan, e, arg
    max
import matplotlib.pyplot as plt

# Importing data
x = array([])
y = array([])
t = array([])

with open('Final Project Video Analysis 161101.csv') as file:
    lines = file.readlines()
    count = 1
    for line in lines:
        if count != 1:
            numbers = line.split(',')
            x = append(x, float(numbers[2]))
            y = append(y, float(numbers[5]))
            t = append(t, float(numbers[0]))
        count += 1

xerr = ones_like(x) * 0.01

h = 0.483
herr = 0.001
yerr = ones_like(y) * 0.005
y1 = h - y
ylerr = herr**2 + yerr**2

# Calculating phi and
phi = arctan(x/y1)
phierr = sqrt((xerr**2 * (y1/(y1+x**2))**2) + (ylerr**2 * (-x/x**2 +
    y1**2)**2))
index = argmax(phierr)

```

```

# Creating a fit line with the parameters
def phifit(A,B,C,l,δ):
    g = 9.8
    fit = (A - B*cos(C*t))*cos(sqrt(g/l)*t - δ)
    return fit

def χ2(phi,phifit,phierr):
    return sum(((phi-phifit)**2)/abs(phifit))

def deriv(parameter,A,B,C,l,δ,Δ):
    f = 0.125
    if parameter == 'A':
        fit1 = phifit(A + (f*Δ),B,C,l,δ)
    elif parameter == 'B':
        fit1 = phifit(A,B + (f*Δ),C,l,δ)
    elif parameter == 'C':
        fit1 = phifit(A,B,C + (f*Δ),l,δ)
    elif parameter == 'l':
        fit1 = phifit(A,B,C,l + (f*Δ),δ)
    else:
        fit1 = phifit(A,B,C,l,δ + (f*Δ))
    fit2 = phifit(A,B,C,l,δ)
    deriv = (χ2(phi,fit1,phierr) - χ2(phi,fit2,phierr))/(f*Δ)
    return deriv

step = 0.0001
ΔA = 0.001
ΔB = 0.001
ΔC = 0.001
Δl = 0.00001
Δδ = 0.001

A = -0.125
B = 0.025
C = 1.9
l = 0.26
δ = 0.1

fit = phifit(A,B,C,l,δ)
oldgrad = 100000000000

```

```

diff = 1

Aarray = array([])
Barray = array([])
Carray = array([])
larray = array([])
δarray = array([])
X2 = array([])

while diff > 1e-30:
    A += step*ΔA*deriv('A',A,B,C,l,δ,ΔA)
    B += step*ΔB*deriv('B',A,B,C,l,δ,ΔB)
    C += step*ΔC*deriv('C',A,B,C,l,δ,ΔC)
    l += step*Δl*deriv('l',A,B,C,l,δ,Δl)
    δ += step*Δδ*deriv('δ',A,B,C,l,δ,Δδ)
    fit = phifit(A,B,C,l,δ)
    newX2 = χ2(phi,fit,phierr)
    grad = sqrt((deriv('A',A,B,C,l,δ,ΔA))**2 +
                (deriv('B',A,B,C,l,δ,ΔB))**2 + (deriv('C',A,B,C,l,δ,ΔC))**2 +
                (deriv('l',A,B,C,l,δ,Δl))**2 + (deriv('δ',A,B,C,l,δ,Δδ))**2)
    diff = oldgrad - grad
    oldgrad = grad
    Aarray = append(Aarray,A)
    Barray = append(Barray,B)
    Carray = append(Carray,C)
    larray = append(larray,l)
    δarray = append(δarray,δ)
    X2 = append(X2,newX2)
    print(f'Minimum X2: {newX2}, Reduced X2: {newX2/(202-4)}')
    print(f'Parameters: A: {A}, B: {B}, C: {C}, l: {l}, δ: {δ}')

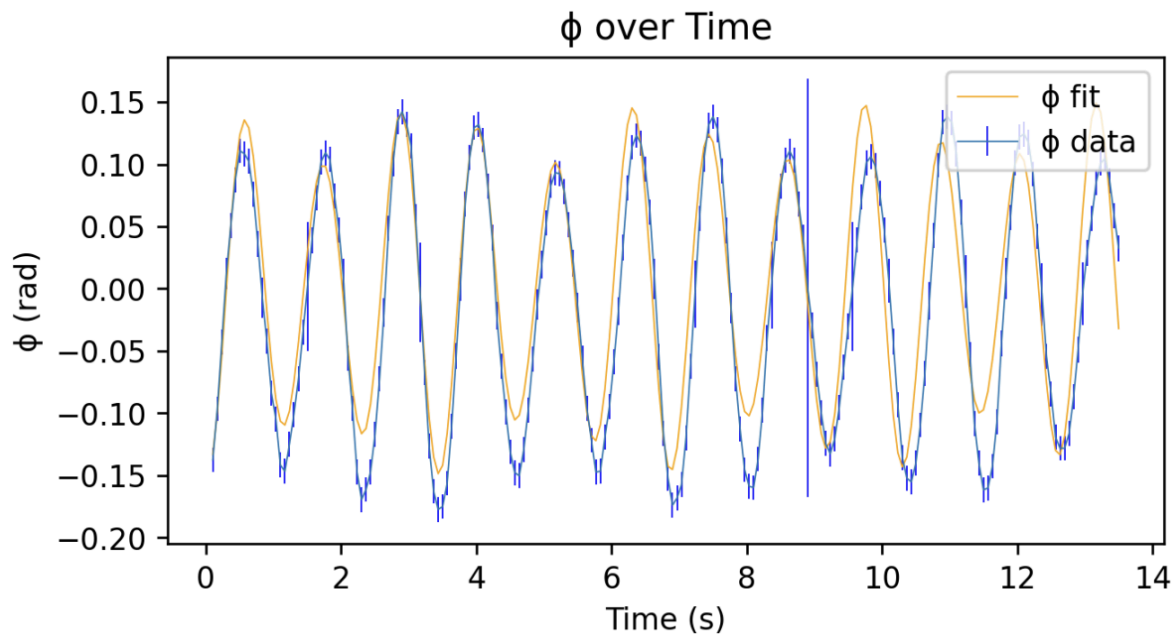
# Calculating numerator and denominator for r formula
num = dot(phi,fit)
den = sqrt(sum(phi**2)) * sqrt(sum(fit**2))

# Calculating r
r = num/den

print(f'Correlation Coefficient: {r}')

```

```
plt.figure(figsize=(6,3),dpi=240)
plt.title('ϕ over Time')
plt.xlabel('Time (s)')
plt.ylabel('ϕ (rad)')
plt.errorbar(t,phi,phierr,ecolor='blue',linewidth=0.5,label='ϕ data')
plt.plot(t,fit,'orange',linewidth=0.5,label='ϕ fit')
plt.legend(loc='upper right')
plt.show()
```



```
from numpy.linalg import det
from numpy import argmin

parameters = ['A','B','C','l','δ']

for i in parameters:
    if i == 'A':
        VarArray = Aarray
    elif i == 'B':
        VarArray = Barray
    elif i == 'C':
        VarArray = Carray
    elif i == 'l':
        VarArray = larray
```

```

else:
    VarArray = δarray

    # Calculating parabola coefficients from last three points
    A1 = (VarArray[-3]**2)
    A2 = (VarArray[-2]**2)
    A3 = (VarArray[-1]**2)
    B1 = VarArray[-3]
    B2 = VarArray[-2]
    B3 = VarArray[-1]
    C1 = X2[-3]
    C2 = X2[-2]
    C3 = X2[-1]

    D1 = array([[A1,B1,1],[A2,B2,1],[A3,B3,1]])
    D2 = array([[C1,B1,1],[C2,B2,1],[C3,B3,1]])
    D3 = array([[A1,C1,1],[A2,C2,1],[A3,C3,1]])
    D4 = array([[A1,B1,C1],[A2,B2,C2],[A3,B3,C3]])

    D = det(D1)
    Da = det(D2)
    Db = det(D3)
    Dc = det(D4)

    a = Da/D
    b = Db/D
    c = Dc/D

    if i == 'A':
        top = -0.12
        bottom = -0.125
    elif i == 'B':
        top = 0.26
        bottom = 0.24
    elif i == 'C':
        top = 2.1
        bottom = 1.8
    elif i == 'l':
        top = 0.326
        bottom = 0.324

```

```

else:
    top = 0.1
    bottom = 0.09

    # Creating a smaller array of variable (Gamma2 in this case) values to
use for the parabola
    Varval = linspace(bottom,top,2000000)
    # Calculating parabola values
    parabola = (a*Varval**2) + (b*Varval) + c

    # Finding the minimum chi squared value and the corresponding variable
value from the parabola
    minX2 = min(parabola)
    x = argmin(parabola)
    finalVar = float(Varval[x])

    # Calculating error in the variable (Gamma2)
    sigmaVar = ((top-bottom)/2000000)*sqrt(2*((X2[-3]**2 - (2*(X2[-2]**2)) +
X2[-1]**2)**-1))

    if i == 'A':
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
A: {finalVar:.7f}, A error: {sigmaVar}')
    elif i == 'B':
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
B: {finalVar:.7f}, B error: {sigmaVar}')
    elif i == 'C':
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
C: {finalVar:.7f}, C error: {sigmaVar}')
    elif i == 'l':
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
l: {finalVar:.7f}, l error: {sigmaVar}')
    else:
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
delta: {finalVar:.7f}, delta error: {sigmaVar}')

```

```

l = 0.26
lerr = 0.002
h = 0.483
herr = 0.001

```

```

yerr = ones_like(y) * 0.005
y1 = h - y
ylerr = herr**2 + yerr**2

r = y1/cos(phi)
epsilon = r - l
epsilonerr = sqrt(lerr**2 + (ylerr**2 * (1/cos(phi))**2) + (phierr**2 *
((y*sin(phi))/(cos(phi))**2)**2))

def epsilon_fit(A,B,C,beta,delta):
    return (A - B*cos(C*t))*cos(sqrt(k/m)*t - delta)*(e**(-beta*t))

def chi2(epsilon,epsilon_fit,epsilonerr):
    return sum(((epsilon-epsilon_fit)**2)/(abs(epsilon_fit)))

def deriv(parameter,A,B,C,beta,delta,delta_step):
    f = 0.125
    if parameter == 'A':
        fit1 = epsilon_fit(A + (f*delta),B,C,beta,delta)
    if parameter == 'B':
        fit1 = epsilon_fit(A,B + (f*delta),C,beta,delta)
    elif parameter == 'C':
        fit1 = epsilon_fit(A,B,C + (f*delta),beta,delta)
    elif parameter == 'beta':
        fit1 = epsilon_fit(A,B,C,beta + (f*delta),delta)
    else:
        fit1 = epsilon_fit(A,B,C,beta,delta + (f*delta))
    fit2 = epsilon_fit(A,B,C,beta,delta)
    deriv = (chi2(epsilon,fit1,epsilonerr) - chi2(epsilon,fit2,epsilonerr))/(f*delta)
    return deriv

step = 0.00001
deltaA = 0.00001
deltaB = 0.00001
deltaC = 0.00001
deltaD = 0.0001
delta_beta = 0.001
delta_delta = 0.001

k = 172

```



```

m = 1

A = 0.035
B = 0.001
C = 0.03
 $\beta$  = 0.00004
 $\delta$  = -0.5

fit =  $\epsilon$ fit(A,B,C, $\beta$ , $\delta$ )
oldgrad = 1000000000000
diff = 1

Aarray = array([])
Barray = array([])
Carray = array([])
 $\beta$ array = array([])
 $\delta$ array = array([])
X2 = array([])

while diff > 1e-6:
    A += step* $\Delta$ A*deriv('A',A,B,C, $\beta$ , $\delta$ , $\Delta$ A)
    B += step* $\Delta$ B*deriv('B',A,B,C, $\beta$ , $\delta$ , $\Delta$ B)
    C += step* $\Delta$ C*deriv('C',A,B,C, $\beta$ , $\delta$ , $\Delta$ C)
     $\beta$  += step* $\Delta$  $\beta$ *deriv('β',A,B,C, $\beta$ , $\delta$ , $\Delta$  $\beta$ )
     $\delta$  += step* $\Delta$  $\delta$ *deriv('δ',A,B,C, $\beta$ , $\delta$ , $\Delta$  $\delta$ )
    fit =  $\epsilon$ fit(A,B,C, $\beta$ , $\delta$ )
    newX2 =  $\chi^2$ ( $\epsilon$ ,fit, $\epsilon$ err)
    grad = sqrt((deriv('A',A,B,C, $\beta$ , $\delta$ , $\Delta$ A))**2 +
(deriv('B',A,B,C, $\beta$ , $\delta$ , $\Delta$ B))**2 + (deriv('C',A,B,C, $\beta$ , $\delta$ , $\Delta$ C))**2 +
(deriv('β',A,B,C, $\beta$ , $\delta$ , $\Delta$ β))**2 + (deriv('δ',A,B,C, $\beta$ , $\delta$ , $\Delta$ δ))**2)
    diff = oldgrad - grad
    oldgrad = grad
    Aarray = append(Aarray,A)
    Barray = append(Barray,B)
    Carray = append(Carray,C)
     $\beta$ array = append( $\beta$ array, $\beta$ )
     $\delta$ array = append( $\delta$ array, $\delta$ )
    X2 = append(X2,newX2)

print(f'Minimum X2: {newX2}, Reduced X2: {newX2/100}')
print(f'Parameters: A: {A} B: {B}, C: {C}, β: {β}, δ: {δ}')

```

```

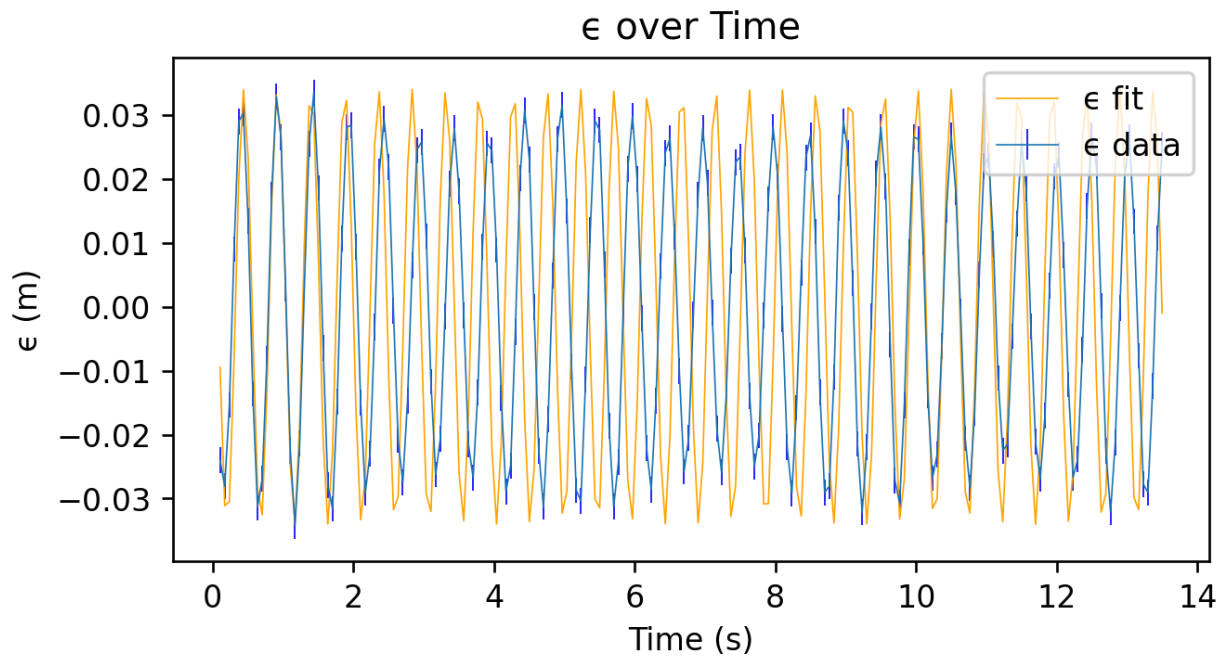
# Calculating numerator and denominator for r formula
num = dot(ϵ,fit)
den = sqrt(sum(ϵ**2)) * sqrt(sum(fit**2))

# Calculating r
r = num/den

print(f'Correlation Coefficient: {r}')

plt.figure(figsize=(6,3),dpi=240)
plt.title('ϵ over Time')
plt.xlabel('Time (s)')
plt.ylabel('ϵ (m)')
plt.errorbar(t,ϵ,ϵerr,ecolor='blue',linewidth=0.5,label='ϵ data')
plt.plot(t,fit,'orange',linewidth=0.5,label='ϵ fit')
plt.legend(loc='upper right')
plt.show()

```



```

from numpy.linalg import det
from numpy import argmin

parameters = ['A','B','C','β','δ']

```

```

for i in parameters:
    if i == 'A':
        VarArray = Aarray
    elif i == 'B':
        VarArray = Barray
    elif i == 'C':
        VarArray = Carray
    elif i == 'β':
        VarArray = βarray
    else:
        VarArray = δarray

    # Calculating parabola coefficients from last three points
    A1 = (VarArray[-3]**2)
    A2 = (VarArray[-2]**2)
    A3 = (VarArray[-1]**2)
    B1 = VarArray[-3]
    B2 = VarArray[-2]
    B3 = VarArray[-1]
    C1 = X2[-3]
    C2 = X2[-2]
    C3 = X2[-1]

    D1 = array([A1,B1,1],[A2,B2,1],[A3,B3,1])
    D2 = array([C1,B1,1],[C2,B2,1],[C3,B3,1])
    D3 = array([A1,C1,1],[A2,C2,1],[A3,C3,1])
    D4 = array([A1,B1,C1],[A2,B2,C2],[A3,B3,C3])

    D = det(D1)
    Da = det(D2)
    Db = det(D3)
    Dc = det(D4)

    a = Da/D
    b = Db/D
    c = Dc/D
    print(a,b,c)

    if i == 'A':

```

```

        top = 0.04
        bottom = 0.03
    elif i == 'B':
        top = 0.003
        bottom = 0.001
    elif i == 'C':
        top = 0.035
        bottom = 0.025
    elif i == 'β':
        top = 4.7e-5
        bottom = 4.2e-5
    else:
        top = -0.4
        bottom = -0.6

    # Creating a smaller array of variable (Gamma2 in this case) values to
    use for the parabola
    Varval = linspace(bottom,top,2000000)
    # Calculating parabola values
    parabola = (a*Varval**2) + (b*Varval) + c

    # Finding the minimum chi squared value and the corresponding variable
    value from the parabola
    minX2 = min(parabola)
    x = argmin(parabola)
    finalVar = float(Varval[x])

    # Calculating error in the variable (Gamma2)
    σVar = ((top-bottom)/2000000)*sqrt(2*((X2[-3]**2 - (2*(X2[-2]**2)) +
X2[-1]**2)**-1))

    if i == 'A':
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
A: {finalVar:.7f}, A error: {σVar}')
    elif i == 'B':
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
B: {finalVar:.7f}, B error: {σVar}')
    elif i == 'C':
        print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}
C: {finalVar:.7f}, C error: {σVar}')

```

```
elif i == 'β':  
    print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}  
β: {finalVar:.10f}, β error: {σVar}')  
else:  
    print(f'Minimum X2: {minX2:.3f}, Reduced X2: {minX2/(202-4):.3f}  
δ: {finalVar:.7f}, δ error: {σVar}')
```